Program 1: Insertion Sort (with Case Analysis via Shift Count)

```
#include <stdio.h>
int A[100], i, j, item, n;
int shift = 0;
float pshift;
int t;
int main()
{
 printf("\n How many numbers: ?");
 scanf("%d", &n);
 printf("\n Enter Elements:");
 for (i = 0; i < n; i++)
 {
 scanf("%d", &A[i]);
 }
 for (j = 1; j < n; j++)
 {
 item = A[j];
 i = j - 1;
 while ((i >= 0) && (item > A[i])) // Note: Use item < A[i] for standard
ascending sort
 {
 A[i + 1] = A[i];
 i = i - 1;
 shift++;
 }
 A[i + 1] = item;
 }
 printf("\n Array elements after sort:");
 for (i = 0; i < n; i++)
 {
```

```
printf("%d\t", A[i]);
}
printf("\n Total shifts: %d\n", shift);
t = n * (n - 1) / 2;
pshift = (float)(shift * 100.0) / t;
if (pshift < 33.0)
{
printf("Best case:");
}
else if (pshift < 66.0)
{
printf("Average case:");
}
else
{
printf("Worst case:");
}
printf("Percentage shift is %f\n", pshift);
return 0;
}
```

Program 2: Quick Sort

```
#include <stdio.h>
int partition(int A[], int low, int high)
{
 int pivot = A[low];
 int i = low + 1;
 int j = high;
 int temp;
 while (1)
 {
 while (i <= high && A[i] <= pivot)
 {
 i++;
 }
 while (A[j] > pivot)
 {
 j--;
 }
 if (i < j)
 {
 temp = A[i];
 A[i] = A[j];
 A[j] = temp;
 }
 else
 {
 break;
 }
 }
 temp = A[low];
 A[low] = A[j];
```

```
 A[j] = temp;
 return j;
}
void quickSort(int A[], int low, int high)
{
 if (low < high)
 {
 int p = partition(A, low, high);
 quickSort(A, low, p - 1);
 quickSort(A, p + 1, high);
 }
}
int main()
{
 int n, i;
 printf("Enter the number of elements: ");
 scanf("%d", &n);
 int A[n];
 printf("Enter %d elements:\n", n);
 for (i = 0; i < n; i++)
 {
 scanf("%d", &A[i]);
 }
 quickSort(A, 0, n - 1);
 printf("\n Sorted Array: ");
 for (i = 0; i < n; i++)
 {
 printf("%d ", A[i]); // Corrected from handwritten scanf-style '&A[i]'
 }
 return 0;
}
```

Program 3: Fractional Knapsack Problem (Greedy Method)

```
#include <stdio.h>
struct obj
{
 int p;
 int w;
 int id;
 float ratio;
 float selected;
};
int main()
{
 struct obj item[10], temp;
 int M, n, i, j;
 int rem;
 float maxp = 0.0;
 float flag[10] = {0};
 float frac;
 printf("Enter the capacity of knapsack: ");
 scanf("%d", &M);
 printf("Enter the number of items: ");
 scanf("%d", &n);
 if (n > 10)
 {
 printf("Maximum number of items is 10.\n");
 return 1;
 }
 for (i = 0; i < n; i++)
 {
 printf("\nEnter profit and weight of item %c: ", i + 65);
 scanf("%d %d", &item[i].p, &item[i].w);
```

```
item[i].id = 65 + i;
item[i].selected = 0.0;
if (item[i].w <= 0)
{
printf("Weight must be greater than 0.\n");
return 1;
}
item[i].ratio = (float)item[i].p / item[i].w;
}
printf("\nEntered Data");
printf("\nItem\tProfit\tWeight\tP/W");
for (i = 0; i < n; i++)
{
printf("\n%c\t%d\t%d\t%.2f", item[i].id, item[i].p, item[i].w, item[i].ratio);
}
// Sorting items in descending order of P/W ratio
for (i = 0; i < n - 1; i++)
{
for (j = 0; j < n - 1; j++)
{
if (item[j].ratio < item[j + 1].ratio)
{
temp = item[j];
item[j] = item[j + 1];
item[j + 1] = temp;
}
}
}
printf("\nSorted Data");
printf("\nItem\tProfit\tWeight\tP/W");
for (i = 0; i < n; i++)
```

```
{
printf("\n%c\t%d\t%d\t%.2f", item[i].id, item[i].p, item[i].w, item[i].ratio);
}
// Knapsack Algorithm
rem = M;
i = 0;
while (rem > 0 && i < n)
{
if (item[i].w <= rem)
{
maxp += item[i].p; // Take whole item
rem = rem - item[i].w;
flag[i] = 1;
}
else
{
maxp += ((float)item[i].p * (float)rem) / (float)item[i].w; // Fraction of
item
frac = (float)(rem * 100) / (float)item[i].w;
flag[i] = frac;
rem = 0; // Knapsack is now full
}
i++;
}
printf("\n >>>> Max Profit Obtained: %.3f\n", maxp);
printf("\n >>>> Item Picked: ");
for (i = 0; i < n; i++)
{
if (flag[i] == 1)
{
printf("%c ", item[i].id);
```

```
 }
 else if (flag[i] > 1)
 {
 printf("%c (%.2f%%) ", item[i].id, flag[i]);
 }
 }
 printf("\n");
 return 0;
}
```

4}MST (minimum spaning tree)

```
#include <stdio.h>
#include <limits.h>
#define MAX 20
int main() {
   int n, i, j;
   int matrix[MAX][MAX];
   int selected[MAX] = {0};   // selected[i] = 1 if vertex i is already in MST
   int edgeCount = 0;
   int minWeight, x, y;
   int totalWeight = 0;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("\nEnter the adjacency matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         printf("Enter value of matrix[%d][%d]: ", i, j);
         scanf("%d", &matrix[i][j]);
      }
   }
   // Start Prim's algorithm from vertex 0
   selected[0] = 1;

   printf("\n");

   while (edgeCount < n - 1) {
      minWeight = INT_MAX;
      x = y = -1;

      // Find the minimum weight edge connecting a selected vertex to an unselected vertex
```

```
        for (i = 0; i < n; i++) {
            if (selected[i]) {
                for (j = 0; j < n; j++) {
                    if (!selected[j] && matrix[i][j] != 0) {
                        if (matrix[i][j] < minWeight) {
                            minWeight = matrix[i][j];
                            x = i;
                            y = j;
                        }
                    }
                }
            }
        }

        if (x != -1 && y != -1) {
            printf("Minimum edge found at %d -> %d with weight %d\n", x, y, minWeight);
            selected[y] = 1;
            totalWeight += minWeight;
            edgeCount++;
        } else {
            break;   // Graph is not connected
        }
    }

    printf("Weight of MST is %d\n", totalWeight);

    return 0;
}
```

5}BFS

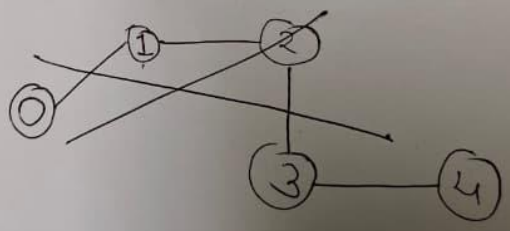
```
#include <stdio.h>
#define Max 20
int adj[Max][Max];
int visited[Max];
int q[Max];
int n;
void bfs(int start)
{
 int front = 0, rear = 0;
 int i, v;
 for (i = 0; i < n; i++)
 {
 visited[i] = 0;
 }
 q[rear++] = start;
 visited[start] = 1;
 printf("BFS traversal: ");
 while (front < rear)
 {
 v = q[front++];
 printf("%d ", v);
 for (i = 0; i < n; i++)
 {
 if (adj[v][i] == 1 && visited[i] == 0)
 {
 q[rear++] = i;
 visited[i] = 1;
 }
 }
 }
```

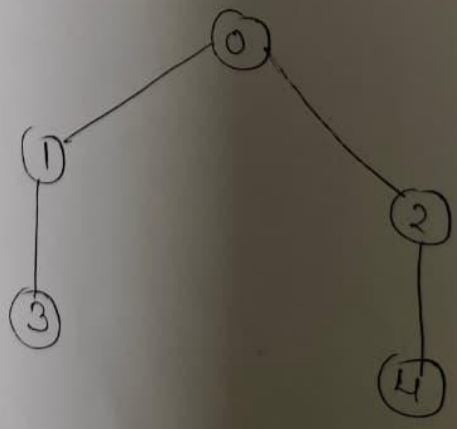
```
 printf("\n");
}
int main()
{
 int i, j, start, edges, u, v;
 printf("Enter number of vertices (max %d): ", Max);
 scanf("%d", &n);
 // Initializing the adjacency matrix to 0
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 {
 adj[i][j] = 0;
 }
 }
 printf("Enter number of edges: ");
 scanf("%d", &edges);
 printf("Enter each edge as two vertex numbers (0-based):\n");
 for (i = 0; i < edges; i++)
 {
 scanf("%d %d", &u, &v);
 adj[u][v] = 1;
 adj[v][u] = 1;
 }
 printf("Enter starting vertex for BFS: ");
 scanf("%d", &start);
 bfs(start);
 return 0;
}
```

Output:-

Output:-

```
Ent no.of vertices (max 20): 5
Ent no.of edges : 4
Enter edges (0-based vertex num):
0 1
0 2
1 3
2 4
Ent starting vertex for Bfs : 0
Bfs Tranversal : 0 1 2 3 4
```





6}0/1KS

```
#include <stdio.h>
int sum = 0;
int max(int a, int b)
{
 if (a > b)
 {
 return a;
 }
 else
 {
 return b;
 }
}
int main()
{
 int w[10], p[10];
 int m, n;
 int v[100][200] = {0};
 int x[10] = {0};
 int i, j;
 printf("Enter number of items\n");
 scanf("%d", &n);
 printf("Enter knapsack capacity\n");
 scanf("%d", &m);
 printf("\nEnter weight and profit of each item:\n");
 for (i = 0; i < n; i++)
 {
 printf("Item %d weight: ", i + 1);
 scanf("%d", &w[i]);
 printf("Item %d profit: ", i + 1);
 scanf("%d", &p[i]);
 }
 // DP table construction
 for (i = 0; i <= n; i++)
 {
 for (j = 0; j <= m; j++)
```

```
{
if (i == 0 || j == 0)
{
v[i][j] = 0;
}
else if (j < w[i - 1])
{
v[i][j] = v[i - 1][j];
}
else
{
v[i][j] = max(v[i - 1][j], v[i - 1][j - w[i - 1]] + p[i - 1]);
}
}
}
// Displaying memoization table
printf("\n Memoization Table:\n\n");
printf("i\\j");
for (j = 0; j <= m; j++)
{
printf("%5d", j);
}
printf("\n");
for (i = 0; i <= n; i++)
{
printf("%3d", i);
for (j = 0; j <= m; j++)
{
printf("%5d", v[i][j]);
}
printf("\n");
}
// Backtracking to find selected items
i = n;
j = m;
while (i > 0 && j > 0)
{
if (v[i][j] != v[i - 1][j])
```

```
{
x[i - 1] = 1;
j = j - w[i - 1];
}
i--;
}
printf("\n Optimal set of items is:\n");
for (i = 0; i < n; i++)
{
if (x[i] == 1)
{
printf("x%d = 1\t", i + 1);
sum = sum + p[i];
}
else
{
printf("x%d = 0\t", i + 1);
}
}
printf("\n Total profit = %d\n", sum);
return 0;
}
```

7}flody warshal

```
#include <stdio.h>
#include <limits.h>

#define MAX 20
#define INF 999

int n;
int D[MAX][MAX];         // Distance matrix
int P[MAX][MAX];         // Pointer (predecessor) matrix
int original[MAX][MAX];   // To keep the original graph

void printMatrix(int mat[MAX][MAX], const char *name) {
   printf("%s:\n", name);
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         if (mat[i][j] == INF)
            printf("%4d", 999);
         else
            printf("%4d", mat[i][j]);
      }
      printf("\n");
   }
   printf("\n");
}
```

```
void printPath(int i, int j) {
    if (i == j) {
        printf("v%d", i + 1);
        return;
    }
    if (P[i][j] == -1) {
        printf("No path");
        return;
    }
    printPath(i, P[i][j]);
    printf(" -> v%d", j + 1);
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter 999 for INF (no direct edge):\n");
    printf("Enter 0 on the diagonal:\n");

    // Read adjacency matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &D[i][j]);
            original[i][j] = D[i][j];
            if (i == j)
```

```
            P[i][j] = -1;         // no predecessor for self
        else if (D[i][j] != INF)
            P[i][j] = i;          // direct edge: predecessor is i
        else
            P[i][j] = -1;         // no path yet
    }
}

// Print original graph edges
printf("\nGraph (directed edges):\n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (i != j && original[i][j] != INF) {
            printf("v%d -> v%d : %d\n", i + 1, j + 1, original[i][j]);
        }
    }
}
printf("\n");

// Print D0 and P0
printMatrix(D, "D 0 Matrix");
printMatrix(P, "P 0 Matrix");

// Floyd-Warshall algorithm
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
            if (D[i][k] != INF && D[k][j] != INF &&
                D[i][k] + D[k][j] < D[i][j]) {
                D[i][j] = D[i][k] + D[k][j];
                P[i][j] = P[k][j];   // update predecessor
            }
        }
    }

    // Print matrices after using vertex k+1 as intermediate
    char dname[20], pname[20];
    sprintf(dname, "D %d Matrix", k + 1);
    sprintf(pname, "P %d Matrix", k + 1);
    printMatrix(D, dname);
    printMatrix(P, pname);
}

// Final shortest paths of all pairs
printf("Shortest paths of all pairs:\n");
printf("All %d ordered pairs, including the %d self-pairs\n\n", n * n, n);

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("v%d to v%d: ", i + 1, j + 1);
        if (i == j) {
            printf("v%d, Distance = 0\n", i + 1);
```

```
            } else if (D[i][j] == INF) {
                printf("No path\n");
            } else {
                printPath(i, j);
                printf(", Distance = %d\n", D[i][j]);
            }
        }
    }

    return 0;
}
```

Op

Enter number of vertices: 5

Enter 999 for INF (no direct edge):

Enter 0 on the diagonal:

0 4 8 999 7

3 0 2 6 999

5 1 0 3 9

999 4 2 0 5

6 999 7 1 0

(also diagram)

8}LCS

```
#include <stdio.h>
#include <string.h>

#define MAX 50

int L[MAX][MAX];         // Length table
char dir[MAX][MAX];      // Direction: 'd' = diagonal, '|' = up, '-' = left

void printTable(char *X, char *Y, int m, int n) {
   // Print column headers (string B)
   printf("\n     ");
   for (int j = 0; j <= n; j++) {
      if (j == 0)
         printf("   ");
      else
         printf("  %c ", Y[j-1]);
   }
   printf("\n");

   // Print first row of zeros
   printf("     ");
   for (int j = 0; j <= n; j++)
      printf("  0 ");
   printf("\n");

   // Print remaining rows
   for (int i = 1; i <= m; i++) {
      printf("  %c  ", X[i-1]);
      printf("  0 ");
      for (int j = 1; j <= n; j++) {
         printf(" %d%c ", L[i][j], dir[i][j]);
      }
      printf("\n");
   }
}
```

```
void printLCS(char *X, char *Y, int m, int n) {
    char lcs[MAX];
    int index = 0;

    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (dir[i][j] == 'd') {
            lcs[index++] = X[i-1];
            i--;
            j--;
        }
        else if (dir[i][j] == '|') {
            i--;
        }
        else {
            j--;
        }
    }

    // Reverse the LCS
    printf("\nThe Character which are same are : ");
    for (int k = index-1; k >= 0; k--)
        printf("%c", lcs[k]);
    printf("\n");

    printf("The length of LCS : %d\n", L[m][n]);
}

int main() {
    char X[MAX], Y[MAX];
    int m, n;

    printf("Enter A String : ");
    scanf("%s", X);
    printf("Enter B String : ");
    scanf("%s", Y);

    m = strlen(X);
```

```
    n = strlen(Y);

    // Initialize first row and column
    for (int i = 0; i <= m; i++)
        L[i][0] = 0;
    for (int j = 0; j <= n; j++)
        L[0][j] = 0;

    // Fill the DP table
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (X[i-1] == Y[j-1]) {
                L[i][j] = L[i-1][j-1] + 1;
                dir[i][j] = 'd';        // diagonal
            }
            else if (L[i-1][j] >= L[i][j-1]) {
                L[i][j] = L[i-1][j];
                dir[i][j] = '|';        // up
            }
            else {
                L[i][j] = L[i][j-1];
                dir[i][j] = '-';        // left
            }
        }
    }

    // Print the table in the same style as your book
    printTable(X, Y, m, n);

    // Print the LCS and its length
    printLCS(X, Y, m, n);

    return 0;
}
```

9}Nqueen

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
int board[20],count,z;
int main()
{
int n,i,j;
int queen(int row,int n);
printf("\n\nEnter number of Queens:");
scanf("%d",&n);
printf("\nNumber of solution required (1-90):");
scanf("%d",&z);
queen(1,n);
return 0;
}
void print(int n)
{
int i,j;
printf("\n\nSolution %d:\n\n",++count);
for(i=1;i<=n;++i)
 printf("\t%d",i);
for(i=1;i<=n;++i)
{
 printf("\n\n%d",i);
 for(j=1;j<=n;++j)
{
 if(board[i]==j)
 printf("\tQ");
 else
 printf("\t.");
 }
}
```

```
}
int place(int row,int column)
{
int i;
for(i=1;i<=row-1;++i)
{
 if(board[i]==column)
 return 0;
 else
 if(abs(board[i]-column)==abs(i-row))
 return 0;
}
return 1;
}
int queen(int row,int n)
{
if(count==z)
return 0;
else
{
int column;
for(column=1;column<=n;++column)
{
 if(place(row,column))
 {
 board[row]=column;
 if(row==n)
 print(n);
 else
 queen(row+1,n);
 }
 }
}  }
```

10}naïve string matching

```
#include <stdio.h>
#include <string.h>
void naiveStringMatch(char text[], char pattern[]) {
 int n = (int)strlen(text);
 int m = (int)strlen(pattern);
 int i, j;
 int found = 0;
 // If pattern is longer than the text, it's an immediate mismatch
 if (m > n || m == 0) {
 printf("Pattern not found\n");
 return;
 }
 for (i = 0; i <= n - m; i++) {
 for (j = 0; j < m; j++) {
 if (text[i + j] != pattern[j]) {
 break;
 }
 }
 if (j == m) {
 printf("Pattern found at index %d\n", i);
 found = 1;
 }
 }
```

```
 if (found == 0) {
 printf("Pattern not found\n");
 }
}
int main() {
 // Large buffers to safely hold long user text inputs
 char text[1000];
 char pattern[100];
 printf("Enter the main text: ");
 fgets(text, sizeof(text), stdin);
 // Remove trailing newline character added by fgets
 text[strcspn(text, "\n")] = '\0';
 printf("Enter the pattern to search: ");
 fgets(pattern, sizeof(pattern), stdin);
 // Remove trailing newline character added by fgets
 pattern[strcspn(pattern, "\n")] = '\0';
 printf("\n--- Search Results ---\n");
 naiveStringMatch(text, pattern);
 return 0;
}
```

OP

```
Enter the main text: AABAACAADAABAABA
Enter the pattern to search: AABA
```